Lab Assignment: The Smart Inventory

Advanced Templates and Memory Management in C++

Computational Physics Course

1. Introduction

In C++, manual memory management using `new` and `delete` is highly susceptible to errors such as memory leaks, dangling pointers, and double-free exceptions. In this lab, you will implement the `Inventory` class, which acts as a **Smart Unique Pointer**. Its primary purpose is to manage a heap resource while ensuring **exclusive ownership** at all times.

2. Learning Objectives

- Implement a Class Template to handle generic types.
- Apply the RAII (Resource Acquisition Is Initialization) principle.
- Enforce **Unique Ownership** by explicitly disabling copy semantics.
- Implement **Move Semantics** to safely transfer resources between objects.
- Utilize **Template Specialization** to achieve polymorphic behavior based on data types.

3. Detailed Instructions

3.1. Step 1: Core Implementation (`Inventory.h`)

Define the template class in the header file. It must contain a private member: `T* ptr`.

1. **Constructor**: Must accept a raw pointer (defaulting to `nullptr`).
2. **Destructor**: Must release the dynamic memory using the `delete` operator.
3. **Access Operators**: Overload `operator*` and `operator->` to simulate native pointer behavior.
4. **Boolean Conversion**: Implement `explicit operator bool() const` to check if the inventory holds a valid resource.

3.2. Step 2: Disabling Copy Semantics

To guarantee exclusivity, you must forbid copying. Declare the Copy Constructor and the Copy Assignment Operator as `delete`. **Experimental Task:** Try to copy an instance in your `main.cc` and document the compiler error.

3.3. Step 3: Move Semantics

Implement the **Move Constructor** and the **Move Assignment Operator**. It is critical to ensure that the *source* object is set to `nullptr` after the transfer to prevent double-free errors when the source object is destroyed.

3.4. Step 4: Specializing `add_item`

The `add_item` function must behave differently depending on the type T:

- **If T is** `float`: The function should add the provided value to the current balance.
- **If T is** `std::vector<std::string>`: The function should append a new string to the end of the vector.

These implementations must be placed in an `Inventory.cc` file using `template <>` syntax.

4. Usage Example

```
int main() {
    // Testing with float
    Inventory<float> wallet(new float(100.0));
    wallet.add_item(float);

    // Testing with vector
    Inventory<std::vector<std::string>> chest(new std::
       vector<std::string>());
    chest.add_item(string);

    // Ownership Transfer
    Inventory<float> newOwner = std::move(wallet);
    return 0;
}
```

5. Verification

Memory integrity will be verified using **Valgrind**. The program must result in 0 errors and 0 bytes lost. Use the following command:

```
valgrind --leak-check=full --show-leak-kinds=all ./your_executable
```